

TUXSHOW V1.5.1: THE OFFICIAL TECHNICAL MANUAL

A Complete Guide for End-Users and IT Administrators in Live Performance

Welcome to TuxShow v1.5.1. If you are reading this, you are likely responsible for ensuring that a live audience experiences a flawless show. In the booth, there is no room for lag, crashed displays, or network intruders.

This manual is written with a strict philosophy: **"Student-proof the booth, protect the show."** We cover everything from low-level audio routing and IPC boundaries to practical script execution and emergency booth communications.

Chapter 1: System Environments & IT Administrator Guide

TuxShow v1.5.1 introduces major architectural designs to guarantee stability across vastly different hardware environments—from beefy gaming laptops to aging school-issued desktops.

1.1 Deployment & The Windows WSLg Framework

While native to Linux, TuxShow runs flawlessly on Windows via the Windows Subsystem for Linux (WSL). For IT administrators deploying on Windows machines, use the standard WSLg graphical architecture. From an elevated PowerShell prompt:

```
bash
wsl --install
```

Once Ubuntu is installed, run the standard TuxShow `.deb` package. WSLg passes the Linux WebGL acceleration directly to the native Windows GPU drivers, ensuring zero-latency projection output.

1.2 The v1.5.1 System Profiler

Educational theater hardware is unpredictable. TuxShow v1.5.1 features a boot-time diagnostic module that scans the host machine's GPU and RAM. It automatically locks the engine into one of three performance tiers to prevent mid-show thermal throttling:

High (60fps): *Requires dedicated GPU. Unlocked multi-layered 1080p WebGL effects.*

Balanced (30fps): Standard for integrated graphics. Perfect for standard crossfades and audio-only playback.

Basic (15fps): Engages automatically on low-RAM devices (e.g., older Raspberry Pis). Prioritizes audio sync and hard-cuts over visual transition smoothing.

1.3 Secure IPC Bridge Boundary

To protect the booth from rendering crashes, v1.5.1 completely isolates the React user interface from the underlying execution systems.

The `coreAppAPI` and `tuxShowAPI` act as an Inter-Process Communication (IPC) bridge. If the UI thread stutters while rendering a complex cue list, the backend `timelineWorker.js` remains completely insulated, ensuring your audio and video cues continue firing with frame-accurate precision.

1.4 Plugin Extensibility System

TuxShow supports third-party extensions via the **Plugin Manager GUI** (accessible by clicking the CPU icon in the top right).

Users can install `.zip` or `.tar.gz` plugins to expand capabilities. To maintain our strict booth-stability philosophy, all plugins operate behind a **Canvas Firewall & Sandboxing** layer. This allows custom React UI tabs to be safely injected into the Inspector without risking access to the core `timelineWorker.js` thread.

Chapter 2: The Master UI & Live Playback Control

The interface is built for the dark. Everything critical is accessible via tactile keystrokes to keep the operator's eyes on the stage, not the screen.

2.1 The Master Control Block (Bottom UI)

Master GO (`<Enter>`): The beating heart of the show. Fires the selected cue and instantly advances the playhead.

Global Pause (`<Spacebar>`): Freezes all active media. Note: Spacebar also engages frame-accurate timeline clock shifting. Re-pressing space resumes playback precisely from the frozen millisecond.

Hard-Stop: Halts a specific cue immediately, dumping it from system memory. If programmed as a transition, it instantly fires the next cue.

Panic Button (`!`): The emergency eject. Executes a global 1.5-second fade-out on all visual and audio elements, bringing the stage to true black and silence.

Record Button (`REC`): Captures the live Master Projection WebGL Canvas along with mixed playback audio (both video and audio cues) and records a native WebM video file directly to the local hard drive. Bypasses booth monitoring states: even if local audio monitoring is muted, the Web Audio API routes the playback streams directly to the WebM file's audio track to ensure a complete visual and auditory archive of the show.

Operator Warning Toasts (New in 1.5.1): Displays instant, floating alert notifications in the bottom-right corner when hardware cues (like webhooks) fail to deliver, ensuring the operator knows of issues instantly

without digging in logs.

[BOOTH COMM] Emergency Mid-Show

SM: "Hold! Actor tripped on the stairs downstage. Video, kill the projections, go to black!"

Operator: "Hitting Panic!" (Operator smashes the  key. The active tornado projection gracefully fades to pure black in 1.5 seconds). "Projections are dark. We are clear for crew."

2.2 Left-Panel Toolbar Utilities

View Toggles: Switch between List View (standard script format) and Timeline View (horizontal multi-track alignment).

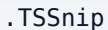
Search/Jump Bar: Rapidly jump to a cue by typing its number.

Unlock All (Master Reset Icon): Instantly clears all runtime locks, resetting the show file to its pre-show state.

Renumber (#): Automatically re-sequences your cue list in increments of 10 (e.g., 10, 20, 30) to allow for late additions (e.g., Cue 15) during tech week.

Add Folder/Cue & Delete: Standard structural commands.

Undo/Redo History: TuxShow features a 50-step undo/redo stack ( / ) that intelligently batches rapid keystrokes to save you from accidental deletions.

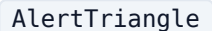
Right-Click Context Menu: Right-clicking on any cue in the list opens a context menu with options to Copy, Paste, or Convert an entire Group into a portable Sequence  file.

2.3 The Inspector & UI Adjustments

Dual-Mode Inspector: The bottom-right Inspector features two tabs. The default **Cue Editor** mode allows you to build the show. Clicking the **Live Media** tab switches the interface to view currently playing cues. Here, operators can scrub playheads, adjust live volume, or hit "Kill" on a specific layer without permanently saving those changes to the show file.

Resizable Panels: Draggable dividers between Cue List, Stage Preview, and Inspector let operators allocate screen space freely (e.g., expanding the Stage Preview for detailed mapping).

Global Master Volume: Scaled globally in the bottom status bar to uniformly limit output independent of individual cue volume values.

Batch Editing (New in 1.5.1): Selecting multiple cues with conflicting values renders a  placeholder styled in italicized, amber font with an  warning icon. Click inside the field to clear and instantly overwrite all values.

Live JSON Validation (New in 1.5.1): Input fields for Webhook headers and bodies validate syntax in real-time. Invalid JSON displays a red border outline and blocks state-saves, protecting the system from corrupt

data writes.

Chapter 3: Audio, Video, & Multi-Projector Topographies

3.1 Chromium Audio Routing (WASAPI/PipeWire)

Nothing ruins a dramatic monologue faster than an OS notification dinging over the PA system. TuxShow bypasses the operating system's default mixer using Chromium's underlying WASAPI (Windows) or PipeWire (Linux) layers.

By utilizing the `setSinkId` protocol, TuxShow forces its audio payload directly to your USB Audio Interface, entirely ignoring the OS default speakers.

3.2 Output Routing & Topographies

Using the **Output Routing** dropdown in the Inspector, operators can dictate exactly where visual media lands on a cue-by-cue basis:

Standalone: One laptop HDMI output hitting a single center-stage projector.

Redundant: A primary and backup laptop, running simultaneously through an A/B hardware video switcher.

Distributed Network: Route cues to specific display targets (e.g., `Display 1` for Stage Left, `Display 2` for Stage Right, or `webrtc` for virtual stream targets). Cues set to `all` will render to all screens.

When projector screens are opened (by clicking **Open Projector Screens** or via settings), TuxShow spawns borderless, full-screen, cursor-free windows mapped directly to each physical output display. A perspective warp grid can be drag-adjusted on each display to map the projection target onto the stage scenery.

3.3 Multi-Projector Stage Preview Tabs (New in 1.5.1)

To monitor what is rendering on different channels from the control console, TuxShow v1.5.1 introduces preview isolation tabs at the top of the **Stage Preview**:

Composite (All): An overlay-composite rendering showing all active visual layers across all outputs merged together.

Individual Displays (e.g., Display 1, Display 2): Dynamically isolates and previews only the visual cues and warping transformations routed to that specific physical display window.

WebRTC (Virtual): Previews the virtual WebRTC stream output broadcast to backend signaling servers.

Chapter 4: Enterprise Network Redundancy & PWA Security

4.1 The Production Wi-Fi Rule

NEVER run TuxShow on a school's public or guest network. You must deploy an isolated, hidden (SSID-broadcast disabled), password-protected router in the booth.

4.2 The Authentication Gateway & Remote PWAs

When broadcasting WebRTC feeds or receiving OSC controls, the system is guarded by a Security PIN gateway. The backend issues a localized `tuxshow_auth` cookie to authenticated booth devices. If an audience member scans your network and tries to access the stream, the Node server drops them instantly with a strict HTTP 401 Unauthorized block.

TuxShow hosts several remote Progressive Web Apps (PWAs) over this secure gateway:

/viewer : A real-time, low-latency Stage Preview monitor.

/camera : Turns a smartphone into a roving, wireless WebRTC camera feed.

/deck : A highly customizable Stage Manager Deck. Provides a grid of colored buttons mapped directly to specific OSC paths for wireless, tablet-based cue triggering.

/buzzer : A specialized remote Game Show Buzzer interface for interactive live events.

[BOOTH COMM] Deploying a Backstage Monitor

SM: "Video, I need eyes on the projections from the Stage Right wing. Can we get the feed on the iPad?"

Operator: "Copy. The iPad is on the hidden show network. Just open Safari and navigate to the `/viewer` endpoint. Enter PIN 4421."

SM: "PIN entered. I have the live stage preview. Looks great."

4.3 Multi-Machine Sync & Redundancy

For high-level professional shows, TuxShow supports seamless "Master" and "Backup" multi-machine topologies:

UDP Telemetry Heartbeat (Port 53001): The Master machine broadcasts a telemetry heartbeat every 1 second. The Backup listens to this heartbeat to confirm connection health.

State Replication (Port 53001): As the operator triggers cues or changes volumes on the Master, the Master broadcasts a UDP state payload. This payload replicates cues, pins, grid size, pause states, and active volumes. To prevent packet fragmentation over UDP, heavy data payloads (such as base64-encoded image mask files) are stripped, relying on the Backup's locally loaded show assets.

TCP Pack Tunnel (Port 53002): When workspace show packages (`.TSPack`) are created or updated, the Master pushes the pack over a TCP HTTP tunnel `/sync-pack` to the Backup. The Backup's HTTP server receives the stream, extracts it using native `tar`, maps the relative paths, and hot-loads it in memory instantly.

Dedicated Receiver Mode: Secondary computers can be locked into a slaved full-screen state (via Settings or `Ctrl+Shift+R`). This spawns a borderless, full-screen, cursor-free WebRTC receiver.

4.4 Settings Diagnostics and Logs Tab

TuxShow provides a real-time debugging panel located in `Settings -> Diagnostics` . This tab displays active telemetry directly from the host system:

Sync States: Shows the current synchronization mode, port binds, UDP socket states, and TCP server status.

Traffic Metrics: Displays total packets sent/received, exact payload size in bytes, and telemetry error logs if a packet is dropped or fails to parse.

Staging Area Telemetry: Details active extraction directories in temp folders (like `/tmp/tuxshow-active-sync`), listing file counts, specific assets, and directory sizes.

System Resource Logs: Real-time console logs and CPU/RAM usage profile meters to detect thermal throttling risks or memory leaks before they affect the show.

Chapter 5: File Management & The Workspace Packer

5.1 The .TSPack Archiving Engine

Moving a 40GB show file from the programming laptop to the booth desktop used to break absolute file paths.

The **Pack Workspace** feature uses the `.TSPack` architecture. Instead of choking the browser with massive files, the Node.js backend spawns a native OS `tar` command via a child process. It bundles everything natively, bypassing browser memory limits. Crucially, it rewrites all absolute file paths (e.g., `C:/Users/Dave/video.mp4`) to portable relative paths (`./media/video.mp4`).

Chapter 6: Comprehensive Cue Types Directory

Understanding how the `timelineWorker.js` evaluates cue behavior at its 60Hz evaluation intervals is critical for complex programming.

6.1 Media Cues

Video:

- *Technical Definition:* Renders 1080p WebM/MP4 frames directly to WebGL textures.
- *Trigger Behavior:* Standard overlap supported. Fade-targets apply to opacity.
- *Real-World Use Case:* Projecting rain falling on a window backdrop.

Audio:

- *Technical Definition:* Hooks directly into `HTMLMediaElement` DOM nodes for `setSinkId` routing.
- *Trigger Behavior:* Hard-stops instantly kill the DOM node. Overlaps allow infinite multi-track layering.
- *Real-World Use Case:* Playing a thunderclap sound effect over a looping ambient rain track.

Image:

- *Technical Definition:* Static 2D projection surfaces loaded into GPU RAM.
- *Trigger Behavior:* Opaque by default. Overlap requires transparency (PNG).
- *Real-World Use Case:* A static title card reading "Act II".

Live Video/Camera:

- *Technical Definition:* Ingests USB Capture Cards, RTSP security feeds, and remote WebRTC `/camera` endpoints. Features automatic frame-freezing disconnect recovery. Includes an advanced WebGL **Chroma Key** filter (Similarity and Smoothness sliders) for real-time green/blue screen removal.

- Trigger Behavior: *Follows standard visual fade rules.*
- Real-World Use Case: *See dialogue below.*

[BOOTH COMM] Managing Wireless Camera Feeds

Director: "For the newsroom scene, I want the actor holding their phone, broadcasting their face live to the main screen."

Operator: "Actor's phone is connected to `/camera` . Firing the Live Video cue now."

SM: "Wait, the actor walked behind the steel set piece, the Wi-Fi dipped!"

Operator: "The engine caught it. It's executing a frame-freeze disconnect recovery. The image is paused, it won't crash... there, Wi-Fi caught up. Video is live again."

Blackout:

- *Technical Definition:* Renders a full-canvas opaque black rectangle over all visual outputs.
- *Trigger Behavior:* Follows opacity fade transitions. Bypasses normal video textures to draw black directly on the projection layer.
- *Real-World Use Case:* Instantly clearing the stage projections to black between scenes while keeping background sound loops running.

6.2 Visual Effects & Shaders

When an active visual cue (Video, Image, or Live Camera) is selected, the **Effects** tab in the Inspector provides access to real-time WebGL shader pipelines.

Performance Warning: Stacking multiple active shaders significantly increases GPU load. If the System Profiler detects severe frame dropping, it will forcefully bypass these shaders to maintain baseline video playback. Proceed with caution when deploying heavily stacked effects on integrated graphics or low-RAM environments.

Color Correction (HSB):

- Capabilities: *Provides individual scalar controls for Hue, Saturation, and Brightness.*
- GUI Location: `Inspector -> Effects Tab -> Live WebGL Filter dropdown -> Color Correction` .
- Cost: *Low overhead. Suitable for global stage tinting or desaturation.*

Blur (Gaussian):

- *Capabilities:* Multi-pass blurring algorithm for depth-of-field simulation. Parameter controls the radius (blur strength).
- *GUI Location:* `Inspector -> Effects Tab -> Live WebGL Filter dropdown -> Blur` .
- *Cost:* High overhead. **Do not stack multiple blur filters.**

Noise / Film Grain:

- Capabilities: *Procedural noise generation over the visual asset. Parameters include Intensity and Speed.*
- GUI Location: `Inspector -> Effects Tab -> Live WebGL Filter dropdown -> Film Grain` .
- Cost: *Moderate overhead.*

Edge Detection (Sobel):

- Capabilities: *Highlights high-contrast edges in the source media, producing a stylized, neon outline effect.*
- GUI Location: `Inspector -> Effects Tab -> Live WebGL Filter dropdown -> Edge Detection` .
- Cost: *High overhead. Useful for stylized transitions or music visualizations.*

Invert:

- Capabilities: *Numerically inverts all RGB values.*
- GUI Location: `Inspector -> Effects Tab -> Live WebGL Filter dropdown -> Invert` .
- Cost: *Minimal overhead.*

6.3 Generative & Rendered Cues

Text / Title:

- *Technical Definition:* Renders dynamic typography directly to the canvas using system fonts.
- *Trigger Behavior:* Fades affect opacity.
- *Real-World Use Case:* Creating rapid lower-thirds with custom formatting, drop-shadows, and alignment for a live broadcast.

Timer:

- *Technical Definition:* *Evaluates elapsed system time to generate real-time canvas clocks. Inherits all Text styling properties.*
- *Trigger Behavior:* *Halts counting on Pause/Hard-Stop.*
- *Real-World Use Case:* *Projecting a 15-minute intermission countdown on the main drape.*

6.4 Animation & Geometry Cues

Animate Cues (Position/Scale):

- *Technical Definition:* Simultaneous translation (`posX` / `posY`) and scale (`scaleX` / `scaleY`) tweens processed by the 60Hz loop.
- *Trigger Behavior:* Hijacks the target asset's properties until the duration completes.
- *Real-World Use Case:* Panning a massive panoramic landscape from left to right, or slowly zooming into a map projection.

Custom Route:

- *Technical Definition:* Translates raw SVG coordinate strings into Bezier motion paths.
- *Trigger Behavior:* Forces the targeted media to strictly adhere to the path timeframe.
- *Real-World Use Case:* Making a projection of a bumblebee fly in a specific looping figure-eight pattern.

6.5 Logic & Control Cues

Group:

- *Technical Definition:* A parent folder. Can evaluate as "Fire-First" (sequential cascade) or "Fire-All" (simultaneous triggers).
- *Trigger Behavior:* Fades apply to the group master bus.
- *Real-World Use Case:* Firing an entire "Pre-Show" sequence with a single click.

Pause (Pause Show):

- *Technical Definition:* Instructs the engine to pause all active media playheads and delay clocks.
- *Trigger Behavior:* Can run as an indefinite pause (waiting for manual operator resume) or with a specific pre-wait duration to automatically resume play.
- *Real-World Use Case:* Pausing a sequence for a live dramatic beat, then resuming when the actor speaks.

Targeted Stop:

- *Technical Definition:* Targets another cue by its cue number and forcibly terminates its playback.
- *Trigger Behavior:* Instantly stops the target cue. If the targeted cue is a Group folder, recursively stops all playing descendant cues inside the group.
- *Real-World Use Case:* Halting a low-level background wind loop cue when entering a new scene.

Conditional (If/Then):

- *Technical Definition:* Branches show execution based on Boolean logic gates. Can evaluate immediate states or run continuously in a 100ms polling worker loop.
- *Trigger Behavior:* Evaluates if a specified cue is playing/stopped/completed, or if an incoming network OSC path matches a target value. Triggers a True Target Cue if correct, or a False Target Cue if incorrect.
- *Real-World Use Case:* Checking if the pre-show music cue is finished; if so, firing the house light dimmers, otherwise waiting.

Sequence (New in 1.5.1):

- *Technical Definition:* A nested timeline container. Child cues evaluate and fire based strictly on their programmed startTime seconds, ignoring standard Auto-Follow logic. Can be imported/exported as portable .TSSnip JSON templates.
- *Trigger Behavior:* Acts as a rigid timeline block. Pausing the sequence pauses all children.
- *Real-World Use Case:* Building a highly choreographed 30-second opening video montage that needs to be portable between different show files.

Select:

- *Technical Definition:* Silently arms a specific cue without triggering it.
- *Trigger Behavior:* Instantly jumps the active selection highlight down the script.
- *Real-World Use Case:* Pre-loading the climax video cue while an ambient audio loop is currently playing.

GoTo:

- *Technical Definition:* Forcibly changes the playhead pointer. Supports Linear jumps or Random range distributions.
- *Trigger Behavior:* Bypasses all intermediate cue logic.
- *Real-World Use Case:* Jumping from the end of Act 1 directly to the Intermission playlist.

Counter:

- *Technical Definition:* Loop tracking and automated sequential progression.
- *Trigger Behavior:* Once the integer hits 0, it auto-follows to the next cue block instead of looping.
- *Real-World Use Case:* Playing an ambient birdsong loop exactly 3 times before moving on.

State-Changer:

- *Technical Definition:* Programmatically enables/disables future script items, or locks assets against Hard-Stops.
- *Trigger Behavior:* Instant background logic execution.
- *Real-World Use Case:* See dialogue below.

Time / Scheduled:

- *Technical Definition:* Evaluates against the host computer's 24-hour system clock.
- *Trigger Behavior:* Auto-fires the target cue when the specific time matches.
- *Real-World Use Case:* Automatically starting a pre-show music playlist at exactly 6:30 PM.

Transition:

- *Technical Definition:* Engages the built-in "Vision Mixer" WebGL shader suite.
- *Trigger Behavior:* Overrides standard alpha fades, applying dynamic wipes (Iris, Star, Curtain, Ripple, Wind) between the outgoing and incoming visual layers.
- *Real-World Use Case:* Executing a classic "Star Wipe" to transition between two game show graphics.

Surtitle (New in 1.5.1):

- *Technical Definition:* Renders sequential lines of subtitles or captions step-by-step. Features a 150ms execution lock to prevent double-skips if manual clicks and auto-advance timers collide.
- *Trigger Behavior:* Advancing transitions lines. Stopping resets playhead index to -1.
- *Real-World Use Case:* Subtitling a foreign-language opera where timing must align with live singing.

IoT Webhook (New in 1.5.1):

- *Technical Definition:* Sends HTTP/HTTPS commands to network targets. Bypasses CORS via native backend bridge execution.
- *Trigger Behavior:* Returns delivery failures as visual error toasts to the operator console.
- *Real-World Use Case:* Triggering smart relays to turn on practical stage lamps during a video cue.

Memo (New in 1.5.1):

- Technical Definition: *visual console notes*.
- Trigger Behavior: *Displays visual instructions inline*.
- Real-World Use Case: *Displays "DANGER: ACTOR ON TRAPDOOR" text box for safety*.

[BOOTH COMM] Rehearsal Reset

Director: "Stop there! The timing was awful. Let's take it back from the top of the act."

Operator: "Standby. Firing the 'Restore Act 1' State-Changer macro. It's defaulting all runtime states, clearing the loop counters, and unlocking the legacy audio tracks. Ready for GO."

6.6 Hardware Integration Cues

OSC: Bi-directional string transmission on **UDP port 53000** (e.g., triggering digital audio consoles).

MSC: Legacy hex-packet macros via ALSA bindings to sync with lighting boards.

DMX Lighting: Art-Net universe broadcasting evaluated at **44Hz** via `dmxEngine.js`. Used to control physical LED fixtures.

Projector Management: Automates physical shutter lenses and projector power states using raw TCP PJLink profiles.

Appendix A: "The Wizard of Oz" Production Walkthrough

To demonstrate TuxShow v1.5.1's capabilities, here is how a community theater tech director programs a complex 7-scene projection layout for The Wizard of Oz*.

1. Scene 1 (The Kansas Farm)

The show opens with a static video loop of a farmhouse. To achieve the classic look, the operator adds a Color Correction effect to the cue, applying a live HSB Filter to instantly drop the visual saturation to 0%, locking in a stark sepia tone.

2. Scene 2 (The Twister)

The operator uses a "Fire-All" Group cue to stack three elements perfectly: A looping tornado video canvas, a heavy wind audio track, and an active Animate Cue. The Animate Cue uses a Custom Route (SVG Bezier motion path) to make a PNG of a cow literally fly across the swirling projection field.

3. Scene 3 (Munchkinland)

As the house lands, the operator fires a 4.0-second Fade-Target cue. This targets the original HSB filter from Scene 1, smoothly crossfading the saturation value from 0% to 100%, blooming the stage into full, vibrant color without cutting the video feed.

4. Scene 4 (The Crossroads)

The yellow brick road is projected onto the floor. To prevent the light from spilling onto the physical cornfield set pieces downstage, the operator uses the Custom Masking polygon tool in the Stage Preview to clip the bottom corners of the projection field.

5. Scene 5 (The Haunted Forest)

To make the forest feel alive, a Continuous Conditional cue is set up. It listens for a live incoming OSC microphone variable from the sound console. Whenever the Wicked Witch speaks into her mic, TuxShow automatically pulses the backdrop's brightness in tandem with her voice.

6. Scene 6 (The Emerald City Gates)

The Emerald City backdrop needs to be projected onto two large, angled physical flats on the stage. The operator activates Mapping Mode and drags the four blue corner pins of a Perspective Warp matrix, squeezing the flat graphic to align perfectly with the physical wood.

7. Scene 7 (The Melting & Return)

As the witch melts, the operator triggers a slow sequence that fades out the active castle elements, unmask the floor, and triggers a reverse Fade-Target. The system comes full circle, returning smoothly to the locked, 0% saturation sepia Kansas farm environment.

Acknowledgments & Licensing

Acknowledgments

With deepest gratitude for years of support, knowledge, and friendship:

Shawna, Madysun, JD, and little Charley (welcome to the world!)

And to my mentors for their guidance:

Tony, Jeff, Mark, Leon, and Squeek!

Licensing (GNU GPLv3)

Copyright (c) 2026 Christopher Earl Baker

TuxShow is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <<https://www.gnu.org/licenses/>>.